

PLANO DE ESTUDOS

Fundamentação Técnica e Revisão Bibliográfica

*Desenvolvimento de uma Plataforma Baseada em Microcontroladores ARM de 32 Bits
para Sistemas de Aquisição de Dados e Simulações HIL*

Projeto PROPG/UEL nº 14615 — LISE, Departamento de Engenharia Elétrica, UEL

Estudante: Pedro de Avance Monteiro

Orientador: Prof. Dr. Daniel Strufaldi Batista

Londrina, 2026

1. Introdução e Objetivo do Plano

Este documento organiza a etapa inicial de fundamentação técnica do projeto, correspondente às Atividades A e B do cronograma de execução do pré-projeto: (A) estudo dos conceitos de sistemas de aquisição de dados (DAQ) e simulações Hardware-in-the-Loop (HIL); e (B) revisão bibliográfica sobre plataformas abertas de aquisição de dados e sistemas embarcados para aplicações em tempo real. O plano assume conhecimento zero nos temas e constrói a base necessária, do conceito mais simples (sinal analógico x digital) até a integração final em uma aplicação HIL, na mesma ordem lógica em que os subsistemas da plataforma serão projetados.

O plano está dividido em 7 blocos de estudo, organizados em um cronograma de 12 semanas (alinhado ao período Set-Nov/2026 previsto no pré-projeto). Cada bloco apresenta os tópicos principais, uma trilha de estudo sugerida, referências específicas (artigos, tutoriais, documentação oficial) e uma sugestão de exercício prático para fixar o conteúdo — essencial em um projeto de hardware, onde teoria sem prática rende pouco.

Uma recomendação de método: mantenha, desde a primeira semana, um documento (ou uma ferramenta como Zotero/Notion) com um resumo de 3-5 linhas por fonte lida, junto da referência completa. Isso evita reler tudo depois na hora de escrever a revisão bibliográfica formal do relatório final.

2. Bloco 0 — Metodologia de Pesquisa e Revisão Bibliográfica

Antes de entrar na parte técnica, vale dominar o processo de pesquisa em si — é o produto central da Atividade B do pré-projeto e a base de qualquer trabalho científico.

2.1 Onde buscar

- Google Scholar — busca ampla, útil para encontrar citações e trabalhos relacionados.
- IEEE Xplore — a maior parte das referências do seu pré-projeto (HIL, DAQ, automação) está aqui.
- ScienceDirect / HardwareX — periódico especializado em hardware científico open-source; várias referências do seu pré-projeto (Oberloier, Pontarolli, Kvalsund, Goertzen, Netzel) vieram daqui. Vale assinar alertas de novos artigos com as palavras-chave do seu tema.
- Portal de Periódicos CAPES — acesso institucional da UEL a bases pagas (Elsevier, IEEE, Springer).

2.2 Como organizar

- Use um gerenciador de referências (Zotero é gratuito e tem plugin para Word/LibreOffice) desde o primeiro artigo lido — organizar depois é muito mais trabalhoso.
- Siga as normas ABNT (NBR 6023 para referências, NBR 10520 para citações) — a UEL normalmente disponibiliza um manual de normalização; vale confirmar com seu orientador se há um template institucional.
- Para cada bloco técnico abaixo, tente localizar pelo menos 1 artigo científico (não apenas tutorial) que trate do tema — isso já começa a construir a revisão bibliográfica do relatório final.

2.3 Recursos

- Google Scholar — <https://scholar.google.com>
- IEEE Xplore — <https://ieeexplore.ieee.org>
- HardwareX (Elsevier/ScienceDirect) — <https://www.sciencedirect.com/journal/hardwarex>
- Zotero (gerenciador de referências gratuito) — <https://www.zotero.org>

3. Cronograma de Estudos — Visão Geral (12 semanas)

O cronograma abaixo cobre o período de estudo intensivo (equivalente a Set–Nov/2026 do pré-projeto). Ele é sequencial: cada bloco depende dos anteriores, então evite pular etapas mesmo que pareçam simples.

Semana	Bloco	Foco da semana
1	Bloco 0 + 6	Metodologia de pesquisa; visão geral de DAQ e HIL (conceitos, Figuras 1–5 do pré-projeto); levantamento de equipamentos comerciais de referência (NI USB-6002, CompactDAQ, dSPACE).
2	Bloco 1	Sinais analógicos x digitais; noções de instrumentação e sensores/transdutores.
3	Bloco 1	Conversores ADC e DAC: resolução, taxa de amostragem, teorema de Nyquist, condicionamento de sinal e filtro anti-aliasing.
4	Bloco 2	Introdução a microcontroladores ARM Cortex-M e à família STM32; toolchain (STM32CubeIDE/CubeMX); GPIO.
5	Bloco 2	Timers, contagem de eventos, geração e medição de PWM em STM32 (prática em kit Nucleo/Discovery, se disponível).
6	Bloco 3	Protocolos seriais: UART, SPI e I2C — teoria e primeiros testes práticos.
7	Bloco 3	Protocolo CAN e fundamentos de USB; USB CDC (Virtual COM Port) no STM32.
8	Bloco 4	Leitura de diagramas elétricos; reguladores de tensão (linear x chaveado); circuitos de proteção elétrica (TVS, ESD).
9	Bloco 4	Introdução ao Altium Designer: captura de esquemático, criação de PCB, geração de Gerber.
10	Bloco 5	Desenvolvimento de firmware em C para aquisição de dados (HAL, interrupções, DMA).
11	Bloco 5	Software supervisor: Python (pyserial + matplotlib) e/ou LabVIEW para visualização e controle em tempo real.
12	Bloco 6	Simulação HIL: conceitos de sistema de tempo real, MATLAB/Simulink Real-Time, dSPACE; consolidação da revisão bibliográfica escrita.

4. Bloco 1 — Sinais e Dados

4.1 Por que começar aqui

Todo o resto do projeto (microcontrolador, protocolos, PCB, firmware) existe para resolver um único problema: capturar um sinal físico (tensão de um sensor, por exemplo) e transformá-lo em número que o computador entenda — e o caminho inverso, gerar um sinal físico a partir de um número. Entender ADC/DAC bem no início evita retrabalho depois.

4.2 Tópicos a dominar

- Diferença entre sinal analógico e digital; o que é quantização.
- Teorema da amostragem de Nyquist — por que a frequência de amostragem precisa ser pelo menos 2x a frequência do sinal.
- Resolução do ADC/DAC (bits) x tensão de referência x menor variação detectável (LSB).
- Arquiteturas de ADC (SAR, sigma-delta, flash) — nível conceitual, não é preciso saber projetar um.
- Condicionamento de sinal: amplificação, filtragem anti-aliasing, isolamento — o que existe entre o sensor e o pino do microcontrolador.
- Noção geral de sensores/transdutores (o que os DAQs do seu laboratório, como a bobina de Helmholtz do LISE, costumam medir).

4.3 Recursos

- A/D Converter: Basic Principle and Types (Utmel) — <https://www.utmel.com/blog/categories/integrated%20circuit/a-d-converter-basic-principle-and-types>
- Analogue to Digital Converter (ADC) Basics — Electronics Tutorials — <https://www.electronics-tutorials.ws/combinational/analogue-to-digital-converter.html>
- Basics of Analog-to-Digital Converters — Arrow — <https://www.arrow.com/en/research-and-events/articles/engineering-resource-basics-of-analog-to-digital-converters>
- Analog Devices Wiki — Cap. 20: Analog to Digital Conversion — <https://wiki.analog.com/university/courses/electronics/text/chapter-20>
- Abdallah & Elkeelany (2009) — A Survey on Data Acquisition Systems DAQ (já referenciado no seu pré-projeto) — <https://doi.org/10.1109/ICC.2009.24>

Exercício sugerido

Pegue um sinal senoidal simples (ex.: saída de um gerador de função ou até um sinal simulado no Excel/Python) e simule o efeito de amostrá-lo com diferentes frequências de amostragem, observando o aliasing quando a taxa fica abaixo de Nyquist. Não precisa de hardware ainda — pode ser feito só em Python com numpy/matplotlib.

5. Bloco 2 — O "Cérebro": Microcontroladores ARM (STM32)

5.1 Tópicos a dominar

- Arquitetura ARM Cortex-M (M0/M3/M4/M7) — diferenças de desempenho e por que o pré-projeto foca em STM32F4/F7.
- Organização de periféricos: clock (RCC), GPIO, barramentos AHB/APB.

- GPIO: modos de entrada/saída, pull-up/pull-down, função alternada (AF).
- Timers: modo contador, captura de entrada (input capture), geração de PWM, encoder.
- Ferramentas: STM32CubeMX (configuração gráfica de periféricos) + STM32CubeIDE (edição/compilação/debug) — fluxo de trabalho padrão da ST.
- Depuração com ST-Link (breakpoints, visualização de registradores).

5.2 Recursos

- Getting Started With STM32 ARM Cortex MCUs — DeepBlueMbedded (trilha completa: GPIO, timers, PWM, ADC) — <https://deepbluembedded.com/getting-started-with-stm32-arm-cortex-mcus/>
- STM32 Timers Tutorial — modos de timer, PWM, interrupções — <https://deepbluembedded.com/stm32-timers-tutorial-hardware-timers-explained/>
- STM32 PWM Output Example Code — <https://deepbluembedded.com/stm32-pwm-example-timer-pwm-mode-tutorial/>
- STM32F103 Blue Pill Tutorial: GPIO, Timer e UART (bom para praticar com placa barata) — <https://zbotic.in/stm32f103-blue-pill-tutorial-gpio-timer-and-uart-guide/>
- Curso "Mastering Microcontroller: Timers, PWM, CAN, Low Power" — notas e exercícios no GitHub — https://github.com/renatosoriano/Udemy-Embedded-Course4_Mastering-Microcontroller-Timers-PWM-CAN-Low-Power-MCU2
- Documentação oficial STM32CubeIDE/CubeMX — ST — <https://www.st.com/en/development-tools/stm32cubeide.html>

Exercício sugerido

Se você tiver acesso a uma placa Nucleo/Discovery STM32F4 (ou mesmo um Blue Pill F103 para praticar o fluxo, já que o hardware final será F4/F7): pisque um LED via GPIO, depois gere um PWM com duty cycle variável, e por fim leia um canal ADC e imprima o valor via UART. Esses três exercícios cobrem os blocos de GPIO, timer/PWM e ADC de uma vez.

6. Bloco 3 — Comunicação: Protocolos e USB

6.1 Tópicos a dominar

- UART: comunicação assíncrona, baud rate, bits de start/stop/paridade — ótimo para debug e comunicação ponto a ponto simples.
- SPI: síncrono, full-duplex, múltiplos escravos via CS — usado quando velocidade importa mais que número de pinos.
- I2C: síncrono, 2 fios, múltiplos dispositivos endereçáveis — comum para sensores.
- CAN: protocolo robusto multi-mestre, usado em automotivo/industrial — citado na metodologia do seu pré-projeto como periférico disponível no STM32F1.
- USB: como funciona a enumeração, diferença entre USB Host/Device, classes USB (a que interessa aqui é a CDC — Communication Device Class).
- USB CDC / Virtual COM Port: como o STM32 aparece como uma porta serial comum no computador sem chip conversor USB-UART externo — essa é a estratégia mencionada no seu pré-projeto para aumentar a taxa de transferência.

6.2 Recursos

- I2C vs SPI vs UART — Introduction and Comparison (TotalPhase) — <https://www.totalphase.com/blog/2021/12/i2c-vs-spi-vs-uart-introduction-and-comparison-similarities-differences/>
- Communication Protocols in Embedded Systems (UART, SPI, I2C, CAN) — <https://easyengineeringhub.in/embedded-communication-protocols-uart-spi-i2c-can-guide/>
- STM32 USB CDC: Virtual COM Port sem UART (HAL) — ControllersTech — <https://controllerstech.com/stm32-usb-cdc-virtual-com-port/>
- STM32 USB CDC Virtual COM Port (VCP) Example — DeepBlueMbedded — <https://deepblueembedded.com/stm32-usb-cdc-virtual-com-port-vcp-examples/>
- Create a USB Virtual COM Port — visão geral de baixo nível do USB CDC — http://janaxelson.com/usb_virtual_com_port.htm

Exercício sugerido

Configure a UART do STM32 para enviar leituras do ADC continuamente para o PC, visualizando no monitor serial. Depois, se possível, repita o mesmo usando USB CDC no lugar da UART, comparando a taxa de transferência prática entre as duas abordagens.

7. Bloco 4 — Hardware e PCB

7.1 Tópicos a dominar

- Leitura de diagramas elétricos: símbolos, referências de componentes (R, C, U, D...), netlist.
- Reguladores de tensão: lineares (LDO, ex. AMS1117/LM7805) x chaveados (buck/boost) — trade-off entre eficiência, ruído e custo.
- Circuitos de proteção elétrica: diodos TVS/ESD nas entradas expostas (USB, conectores externos), fusíveis rearmáveis (PTC).
- Alimentação dupla (USB + fonte externa) — mencionada explicitamente na arquitetura do seu pré-projeto.
- Fluxo de projeto em ferramenta EDA: esquemático → PCB → geração de Gerber para fabricação.

7.2 Recursos

- Altium Designer — Quick-Start Tutorial (fluxo completo em ~25 min) — <https://resources.altium.com/p/altium-designer-quick-start-tutorial-part-1>
- Getting Started with PCB Design — tutorial oficial Altium — <https://resources.altium.com/p/getting-started-pcb-design>
- How to Create a PCB Schematic in Altium Designer — <https://resources.altium.com/p/schematic-tutorial-altium-designer-journey-thousand-pcbs>
- Altium Designer Tutorial for Beginners — passo a passo (PCBSync) — <https://pcbsync.com/altium-designer-tutorial-for-beginners/>
- Beginner's Guide to ESD Protection Circuit Design for PCBs — Altium — <https://resources.altium.com/p/beginners-guide-esd-protection-circuit-design-pcbs>
- PCB Design Guidelines for Using TVS Diode for Transient Protection — Altium — <https://resources.altium.com/p/pcb-design-guidelines-using-tvs-diode-transient-protection>

Exercício sugerido

Antes de desenhar a placa final do projeto, refaça um circuito simples no Altium (ex.: um regulador 5V→3,3V com capacitores de desacoplamento e um diodo TVS na entrada USB) só para praticar o fluxo completo: esquemático, footprints, roteamento e exportação de Gerber.

8. Bloco 5 — Programação: Firmware e Software Supervisório

8.1 Tópicos a dominar

- Firmware em C: estrutura de um projeto STM32 HAL, uso de interrupções, DMA para transferência de dados sem sobrecarregar a CPU.
- Boas práticas de firmware para DAQ: buffers circulares, timestamping das amostras, protocolo de comunicação simples entre firmware e PC (ex.: pacotes com cabeçalho + checksum).
- Python para instrumentação: pyserial para comunicação serial, matplotlib para visualização em tempo real, pandas/CSV para registro de dados.
- LabVIEW (alternativa mencionada no pré-projeto): instrumento virtual (VI), front panel x block diagram, aquisição via VISA/porta serial.

8.2 Recursos

- Real Time Serial Data Monitor with Python (PySerial + Matplotlib) — <https://downonearthtutorials.blogspot.com/2014/09/real-time-serial-data-monitor-with.html>
- Plotting Live Data in Real-Time with Python using Matplotlib — <https://datacapturecontrol.com/articles/data-acquisition/computer-software/live-plotting-matplotlib>
- serial_plotter — exemplo de código Python + PySerial + Matplotlib (GitHub) — https://github.com/aerialist/serial_plotter
- PyRealtime — pipeline simplificado para plotagem serial em tempo real — <https://pypi.org/project/PyRealtime>
- Kvalsund et al. (2023) — Development of an Arduino-based, open-control interface for HIL applications (HardwareX, já no seu pré-projeto) — <https://doi.org/10.1016/j.ohx.2023.e00488>

Exercício sugerido

Feche o ciclo completo pela primeira vez: firmware no STM32 lê um canal ADC e envia via serial/USB; script em Python recebe, plota em tempo real e salva em CSV. Esse é essencialmente um DAQ mínimo funcionando — o mesmo princípio será expandido no projeto real.

9. Bloco 6 — Simulação Hardware-in-the-Loop (HIL)

9.1 Tópicos a dominar

- O que é simulação em tempo real e por que HIL é diferente de uma simulação offline comum.
- Arquitetura básica: modelo computacional (plant) rodando em tempo real + hardware real trocando sinais com esse modelo.
- Categorias de HIL (Controller-HIL, Power-HIL) — nível conceitual.
- Plataformas comerciais de referência citadas no pré-projeto: MATLAB/Simulink Real-Time, dSPACE, além dos DAQs da NI (USB-6002, CompactDAQ/cRIO) — vale entender o que cada uma oferece e por que têm custo elevado.
- Como a plataforma do projeto se encaixa: atua como a "interface física" entre o modelo simulado (rodando no PC/host) e dispositivos eletrônicos externos.

9.2 Recursos

- What Is Hardware-in-the-Loop (HIL)? — MathWorks (excelente introdução, direto ao ponto) — <https://www.mathworks.com/discovery/hardware-in-the-loop-hil.html>
- What is Hardware-in-the-Loop Testing? — Ansys — <https://www.ansys.com/simulation-topics/what-is-hardware-in-the-loop-testing>
- An Introduction to Hardware-in-the-loop (HIL) Simulation and Its Applications — https://www.academia.edu/28664861/An_Introduction_to_Hardware_in_the_loop_HIL_Simulation_and_Its_Applications
- Schlager (2008) — Hardware-in-the-Loop Simulation (livro-base já citado no seu pré-projeto) — <https://www.springer.com>
- Kvalsund et al. (2023) — plataforma HIL open-source baseada em Arduino (comparável, em escala menor, ao que você vai construir) — <https://doi.org/10.1016/j.ohx.2023.e00488>

Exercício sugerido

Como fechamento do bloco (e do plano de estudos como um todo), monte um mini relatório comparativo de 1-2 páginas: para cada equipamento comercial estudado (USB-6002, CompactDAQ, dSPACE), liste custo aproximado, principais recursos e limitações — esse material alimenta diretamente a seção de justificativa/estado da arte do relatório final.

10. Recomendações Finais

- Estude sempre com a mão na massa: leitura sem prática em kits/simuladores rende pouca retenção nesses temas.
- Vá anotando, desde a Semana 1, cada fonte lida em um gerenciador de referências — isso poupa semanas de trabalho na hora de escrever a revisão bibliográfica formal.
- Revise este cronograma com seu orientador a cada 2–3 semanas; ele pode reordenar prioridades conforme o hardware disponível no LISE.
- Ao final da Semana 12, você deve ser capaz de: explicar a arquitetura de um DAQ, justificar a escolha do STM32F4/F7, escolher entre UART/SPI/I2C/CAN/USB para uma dada aplicação, entender um esquemático simples e produzir um protótipo mínimo de aquisição serial em Python — base suficiente para iniciar a Atividade C (levantamento de requisitos e arquitetura da plataforma).

Bons estudos — e qualquer bloco específico que você quiser aprofundar (por exemplo, entrar mais fundo em USB CDC ou em condicionamento de sinal), é só pedir que detalhamos em uma sessão de estudo dedicada.